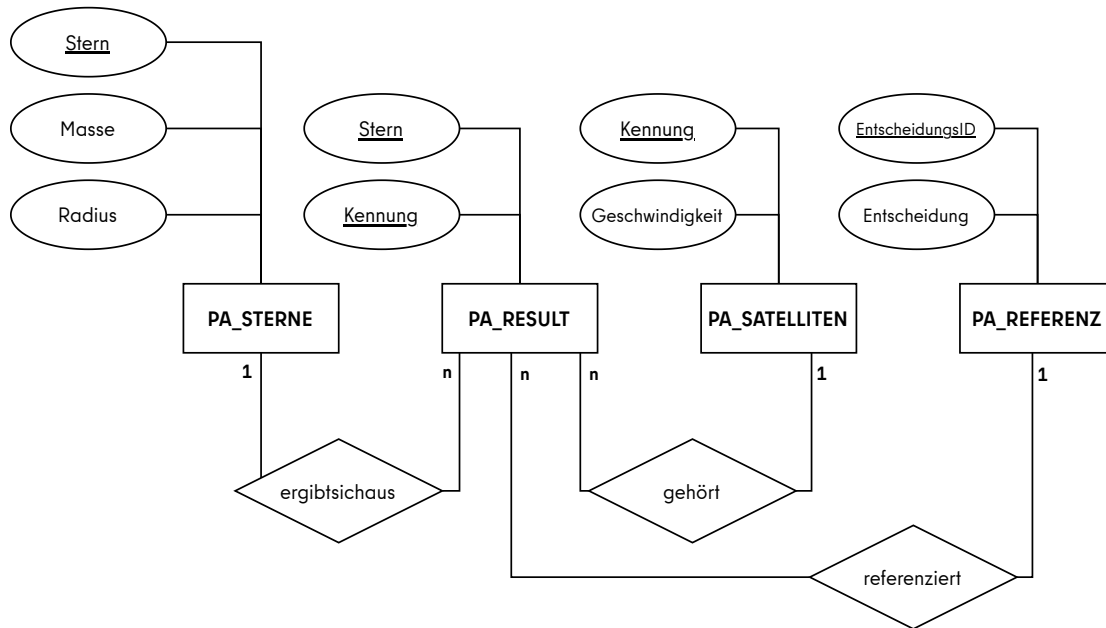


Aufgabe 1:

ERM in Peter-Chen-Notation



Aufgabe 2:

ERM als Tabellen

<u>Stern</u>	Masse	Radius
Aldebaran	3.38e30	3.07e10
Arktur	2.19e30	1.77e10
Betelgeuse	3.28e31	6.17e11
Orion	6.20e35	1.67e13
Polarstern	8.70e30	7.78e08
Sonne	1.99e30	6.96e08
Erde	5.97e24	6.37e06

Tabelle 1: **PA_STERNE**

<u>Kennung</u>	Geschwindigkeit
Bohr	9.90e04
Galileo	5.00e05
Higgs	1.28e14
Kopernikus	1.31e08
Newton	9.10e03
Plank	7.77e78

Tabelle 2: **PA_SATELLITEN**

<u>EntscheidungsID</u>	Entscheidung
0	Kreisen
1	Kollidieren
2	Weiter fliegen
9	Entscheidungsfehler

Tabelle 3: **PA_REFERENZ**

<u>Stern</u>	<u>Kennung</u>	<u>EntscheidungsID</u>
Orion	Plank	2
...	...	...

Tabelle 4: **PA_RESULT**



Aufgabe 3:

Normalformen der Tabellen

PA_STERNE

Die Tabelle **PA_STERNE** hat den Primärschlüssel **Stern** und die Attribute **Masse** sowie **Radius**. Da alle Werte atomar vorliegen, ist die 1NF erfüllt, und wegen des einattributigen Schlüssels ist auch die 2NF unmittelbar gegeben.

Masse und **Radius** werden jeweils direkt durch **Stern** bestimmt; eine transitive Abhängigkeit liegt nicht vor, weil die Nichtschlüsselattribute einander nicht funktional bestimmen. Damit befindet sich **PA_STERNE** in der dritten Normalform.

PA_SATELLITEN

Für **PA_SATELLITEN** mit **Kennung** als Primärschlüssel und **Geschwindigkeit** als einzigem Nichtschlüsselattribut gilt dieselbe Logik. Die Attributwerte sind atomar (1NF), und partielle Abhängigkeiten sind durch den einteiligen Schlüssel ausgeschlossen (2NF).

Da **Geschwindigkeit** ausschließlich von **Kennung** abhängt und kein weiteres Nichtschlüsselattribut vorhanden ist, kann keine transitive Abhängigkeit entstehen. Die Tabelle erfüllt somit die 3NF.

PA_REFERENZ

Die Tabelle **PA_REFERENZ** besteht aus dem Primärschlüssel **EntscheidungsID** und dem Attribut **Entscheidung**. Auch hier sind die Werte atomar und wegen des einattributigen Schlüssels treten keine partiellen Abhängigkeiten auf.

Entscheidung hängt direkt von **EntscheidungsID** ab, und ein transitiver Pfad ist mangels weiterer Nichtschlüsselattribute nicht möglich. Daher liegt **PA_REFERENZ** ebenfalls in der dritten Normalform.

PA_RESULT

PA_RESULT unterscheidet sich durch den zusammengesetzten Primärschlüssel aus **Stern** und **Kennung**. Das Nichtschlüsselattribut **EntscheidungsID** wird erst durch die Kombination beider Schlüsselbestandteile festgelegt; weder **Stern** allein noch **Kennung** allein reicht dafür aus.

Damit ist die zweite Normalform erfüllt, weil keine partielle Abhängigkeit vorliegt. Da **EntscheidungsID** das einzige Nichtschlüsselattribut ist, sind außerdem keine transitiven Abhängigkeiten möglich, sodass auch **PA_RESULT** in der 3NF liegt.



Aufgabe 4:

SQL Tabelleninitialisierung

```
BEGIN EXECUTE IMMEDIATE 'DROP TABLE PA_RESULT CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE PA_SATELLITEN CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE PA_STERNE CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE PA_REFERENZ CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
CREATE TABLE PA_REFERENZ (
    EntscheidungID INTEGER PRIMARY KEY,
    Entscheidung VARCHAR2(50) NOT NULL
);

CREATE TABLE PA_STERNE (
    Stern VARCHAR2(50) PRIMARY KEY,
    Masse NUMBER NOT NULL,
    Radius NUMBER NOT NULL
);

CREATE TABLE PA_SATELLITEN (
    Kennung VARCHAR2(50) PRIMARY KEY,
    Geschwindigkeit NUMBER NOT NULL
);

CREATE TABLE PA_RESULT (
    Stern VARCHAR2(50) NOT NULL,
    Kennung VARCHAR2(50) NOT NULL,
    EntscheidungID INTEGER NOT NULL,
    CONSTRAINT pk_result PRIMARY KEY (Stern, Kennung),
    CONSTRAINT fk_result_stern FOREIGN KEY (Stern)
        REFERENCES PA_STERNE (Stern),
    CONSTRAINT fk_result_sat FOREIGN KEY (Kennung)
        REFERENCES PA_SATELLITEN (Kennung),
    CONSTRAINT fk_result_ref FOREIGN KEY (EntscheidungID)
        REFERENCES PA_REFERENZ (EntscheidungID)
);

COMMIT;

INSERT ALL
    INTO PA_REFERENZ (EntscheidungID, Entscheidung) VALUES (0, 'Kreisen')
    INTO PA_REFERENZ (EntscheidungID, Entscheidung) VALUES (1, 'Kollidieren')
    INTO PA_REFERENZ (EntscheidungID, Entscheidung) VALUES (2, 'Weiter fliegen')
    INTO PA_REFERENZ (EntscheidungID, Entscheidung) VALUES (9, 'Entscheidungsfehler')
SELECT 1 FROM dual;

INSERT ALL
    INTO PA_STERNE (Stern, Masse, Radius) VALUES ('Aldebaran', 3.38E+30, 3.07E+10)
    INTO PA_STERNE (Stern, Masse, Radius) VALUES ('Arktur', 2.19E+30, 1.77E+10)
    INTO PA_STERNE (Stern, Masse, Radius) VALUES ('Betelgeuse', 3.28E+31, 6.17E+11)
    INTO PA_STERNE (Stern, Masse, Radius) VALUES ('Orion', 6.20E+35, 1.67E+13)
    INTO PA_STERNE (Stern, Masse, Radius) VALUES ('Polarstern', 8.70E+30, 7.78E+08)
    INTO PA_STERNE (Stern, Masse, Radius) VALUES ('Sonne', 1.99E+30, 6.96E+08)
    INTO PA_STERNE (Stern, Masse, Radius) VALUES ('Erde', 5.97E+24, 6.37E+06)
SELECT 1 FROM dual;

INSERT ALL
    INTO PA_SATELLITEN (Kennung, Geschwindigkeit) VALUES ('Bohr', 9.90E+04)
    INTO PA_SATELLITEN (Kennung, Geschwindigkeit) VALUES ('Galileo', 5.00E+05)
    INTO PA_SATELLITEN (Kennung, Geschwindigkeit) VALUES ('Higgs', 1.28E+14)
    INTO PA_SATELLITEN (Kennung, Geschwindigkeit) VALUES ('Kopernikus', 1.31E+08)
    INTO PA_SATELLITEN (Kennung, Geschwindigkeit) VALUES ('Newton', 9.10E+03)
    INTO PA_SATELLITEN (Kennung, Geschwindigkeit) VALUES ('Plank', 7.77E+78)
SELECT 1 FROM dual;

COMMIT;
```

Code 1: [code/A4.sql](#)



Aufgabe 5:

Paketspezifizierung

```
CREATE OR REPLACE PACKAGE SAT AS
  C_GRAVITATIONSKONSTANTE CONSTANT NUMBER := 6.67E-11;
  C_WURZEL_ZWEI          CONSTANT NUMBER := 1.41421356237;

  PROCEDURE Action;
END SAT;
/
```

Code 2: **code/A5.sql**

Aufgabe 6/7:

Implementierung des Paketbodys

```
CREATE OR REPLACE PACKAGE BODY SAT AS

  PROCEDURE GetVelocity(
    p_EKG OUT NUMBER,
    p_ZKG OUT NUMBER,
    p_Masse IN NUMBER,
    p_Radius IN NUMBER
  ) IS
  BEGIN
    p_EKG := SQRT((C_GRAVITATIONSKONSTANTE * p_Masse) / p_Radius);
    p_ZKG := C_WURZEL_ZWEI * p_EKG;
  END GetVelocity;

  PROCEDURE Action IS
    v_EKG NUMBER;
    v_ZKG NUMBER;
    v_EntscheidungID INTEGER;

    CURSOR c_Sterne IS
      SELECT Stern, Masse, Radius
      FROM PA_STERNE;

    CURSOR c_Satelliten IS
      SELECT Kennung, Geschwindigkeit
      FROM PA_SATELLITEN;
  BEGIN
    FOR r_Stern IN c_Sterne LOOP
      FOR r_Satellit IN c_Satelliten LOOP
        GetVelocity(v_EKG, v_ZKG, r_Stern.Masse, r_Stern.Radius);

        IF r_Satellit.Geschwindigkeit < v_EKG THEN
          v_EntscheidungID := 1;
        ELSIF r_Satellit.Geschwindigkeit <= v_ZKG THEN
          v_EntscheidungID := 0;
        ELSIF r_Satellit.Geschwindigkeit > v_ZKG THEN
          v_EntscheidungID := 2;
        ELSE
          v_EntscheidungID := 9;
        END IF;

        INSERT INTO PA_RESULT (Stern, Kennung, EntscheidungID)
        VALUES (r_Stern.Stern, r_Satellit.Kennung, v_EntscheidungID);
      END LOOP;
    END LOOP;
  END Action;

END SAT;
/
```

Code 3: **code/A6-7.sql**



Aufgabe 8:

Anonymer Block zur Testausführung der Prozedur Action

```
DECLARE
  CURSOR c_Ergebnis IS
    SELECT r.Stern,
           r.Kennung,
           ref.Entscheidung
    FROM PA_RESULT r
    JOIN PA_REFERENZ ref
    ON r.EntscheidungID = ref.EntscheidungID
    ORDER BY r.Stern, r.Kennung;
BEGIN
  DELETE FROM PA_RESULT;

  SAT.Action;

  FOR r_Ergebnis IN c_Ergebnis LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Stern: ' || RPAD(r_Ergebnis.Stern, 12) ||
      ' | Satellit: ' || RPAD(r_Ergebnis.Kennung, 12) ||
      ' | Entscheidung: ' || r_Ergebnis.Entscheidung
    );
  END LOOP;
END;
/
COMMIT;
```

Code 4: **code/A8.sql**